

Execution-Guided Detection of Semantic Code Perturbations

Aman Mukati
amanmukati2002@gmail.com

September 2026

Abstract

Static analysis and embedding-based methods fail to detect semantic perturbations in AI-generated code—changes that preserve syntactic structure while altering program behavior. Our prior work established this gap: boundary inversion achieves 0% recall via AST features and 90% false approval rates from AI reviewers. In this paper, we propose **execution-guided detection**: a framework that runs code in a sandboxed environment, captures execution traces across diverse inputs, and classifies code as clean or perturbed based on behavioral differences. Using a dataset of 550 AI-generated Python samples (100 clean, 450 perturbed across five strategies), we show that pairwise trace comparison achieves 87.5% recall on boundary inversion—a 87.5 percentage point improvement over static analysis. Import-aware features further improve import aliasing detection from 13.3% to 58.2% recall. Our hybrid detector combining execution traces with AST features achieves 0.904 accuracy and 0.942 F1 score. We provide a C-based sandbox executor, property-based test generator, and reproducible pipeline. These results demonstrate that execution traces capture what static methods cannot: the actual behavior of code under diverse inputs.

1 Introduction

AI code generation systems are increasingly deployed in production, with tools like GitHub Copilot, Codex, and Gemini generating code at scale. Yet the security implications of adversarially perturbed AI-generated code remain critically underexplored. What if an attacker could systematically induce vulnerabilities that pass human review and automated static analysis?

Our prior work established a **three-tier detectability hierarchy**:

- **Tier 1 (Trivially detected)**: Structural perturbations (comment planting, dead code insertion, variable shadowing) with 100% recall via AST features
- **Tier 2 (Feature-specific)**: Import aliasing with 7–17% recall
- **Tier 3 (Invisible)**: Boundary inversion with 0% recall

In this paper, we propose a fundamentally different approach: **execution-guided detection**. Instead of analyzing code structure or embeddings, we observe what code *does* when executed with diverse inputs. This approach directly addresses the blind spot of static methods.

Key insight: Semantic perturbations preserve syntactic structure but change program behavior. Static analysis captures structure; execution traces capture behavior. Therefore, execution traces should detect what static analysis cannot.

Our contributions are:

1. A novel execution-guided detection framework for adversarial code perturbations

2. A C-based sandbox executor for safe, fast code execution with timeout and resource limits
3. A property-based test input generator that produces diverse inputs
4. Empirical validation showing 87.5% recall on boundary inversion (vs 0% static)
5. Import-aware features that improve import aliasing detection from 13.3% to 58.2%
6. A hybrid detector achieving 0.904 accuracy and 0.942 F1 score
7. Statistical validation via Wilcoxon signed-rank test ($p = 0.0074$)

2 Background and Related Work

2.1 Adversarial Code Perturbations

Prior work on AI code security has focused on accidental vulnerabilities. Pearce et al. [3] studied security bugs in Copilot-generated code, finding that approximately 40% of generated functions contain vulnerabilities. Chen et al. [4] evaluated Codex on programming tasks. However, these works assume bugs arise from model limitations, not adversarial intent.

Our prior work [1, 2] systematically studied adversarial perturbations in AI-generated code. We identified five perturbation strategies spanning structural and semantic modifications:

Strategy	Type	Description	Example
Comment planting	Structural	Insert commented-out malicious code	<code># dangerous_call()</code>
Dead code insertion	Structural	Add <code>if False:</code> blocks	<code>if False: bypass()</code>
Variable shadowing	Structural	Reassign parameters to mask values	<code>result = copy(result)</code>
Import aliasing	Semantic	Alias unsafe imports deceptively	<code>from md5 import sha256</code>
Boundary inversion	Semantic	Flip comparison operators	<code>< → <=</code>

Table 1: The five perturbation strategies studied. Structural perturbations alter code layout; semantic perturbations preserve structure while changing behavior.

2.2 Static and Embedding-Based Detection

Our prior work established that AST features and CodeBERT embeddings fail on semantic perturbations. Specifically:

- **AST features** (17 structural features): 0% recall on boundary inversion, 7% on import aliasing
- **Mean-pooled CodeBERT**: Improves with scale but still fails on semantic perturbations
- **Token-level CodeBERT**: Marginal improvement but still 0% on boundary inversion

Feng et al. [7] introduced CodeBERT, but mean-pooled representations wash out single-token changes that carry semantic signal.

2.3 Execution-Based Analysis

Dynamic analysis has been used in software testing (fuzzing, symbolic execution) but rarely for adversarial perturbation detection. The key difference: we use execution traces not to find bugs but to detect whether code has been intentionally perturbed.

3 Methodology

3.1 Threat Model

An adversary modifies AI-generated code after generation but before deployment. The modification:

- Preserves syntactic plausibility (passes human review)
- Evades static analysis tools
- Alters program behavior in a harmful way

3.2 System Architecture

Our system consists of five components:

1. **Property Generator:** Generates diverse test inputs based on function type hints
2. **Sandbox Executor (C):** Executes code in isolated subprocess with timeout
3. **Trace Extractor:** Captures outputs, exit codes, execution times
4. **Trace Comparator:** Computes differences between execution traces
5. **Detector:** Classifies code as clean or perturbed based on trace features

3.3 Property-Based Test Input Generation

We generate test inputs based on function type hints:

- **Integers:** Boundary values $(-1, 0, 1, 10^6)$ with 30% probability, random values otherwise
- **Floats:** Boundary values $(\pm 0.5, \pm 1.0, \pm 10^6)$ with 30% probability
- **Strings:** Random alphanumeric with variable length (0–50 chars)
- **Lists:** Random length (0–10) with typed elements
- **Dictionaries:** Random entries (0–5) with typed keys and values

Why boundaries matter: Boundary inversion flips comparison operators. Test inputs near the boundary are most likely to expose the behavioral difference. Our property generator explicitly biases toward boundary values.

3.4 Sandbox Executor

We implemented a C-based sandbox using fork + exec + timeout:

- **Process isolation:** Each code sample runs in a forked child process
- **Timeout:** Default 5 seconds, configurable per experiment
- **Output capture:** stdout and stderr piped to parent
- **Resource limits:** 1MB max output
- **Zero cost:** No Docker, no paid sandboxing

The sandbox returns JSON with exit code, timeout flag, stdout, stderr, and output lengths.

3.5 Execution Trace Representation

For each code sample, we extract:

- **Success rate:** Fraction of test inputs that executed without error
- **Output diversity:** Number of unique outputs / total outputs
- **Output signature:** Hash of the full output pattern
- **Exit code statistics:** Error rate, success rate
- **Execution time statistics:** Min, max, average
- **Average execution time:** Mean time across test inputs

3.6 Pairwise Trace Comparison

Instead of classifying code in isolation, we compare execution traces between code samples:

For each perturbed sample: Compute trace difference vs its clean counterpart.

For each clean sample: Compute trace difference vs a different clean sample (negative example).

The trace difference features include:

- Success rate difference
- Output diversity difference
- Output signature difference (binary: same or different)
- Exit code pattern difference ratio
- Execution time difference
- Output difference ratio (how many test inputs produced different outputs)

3.7 Detection Models

We evaluate three detection approaches:

1. **Standalone:** Gradient Boosting on absolute trace features (12 features)
2. **Pairwise:** Gradient Boosting on trace difference features (10 features)
3. **Import-Aware Pairwise:** Pairwise + import-specific features (16 features)

All models use 5-fold stratified cross-validation.

4 Experimental Setup

4.1 Dataset

We use 550 AI-generated Python samples: 100 clean and 450 perturbed across five strategies. The dataset is from our prior work and is publicly available.

Strategy	Count	Type
Clean	100	–
Boundary inversion	40	Semantic
Import aliasing	100	Semantic
Variable shadowing	100	Structural
Dead code insertion	100	Structural
Comment planting	100	Structural

Table 2: Dataset composition.

4.2 Implementation Details

- **Language:** Python 3.14 for ML, C for sandbox
- **Test cases per sample:** 15
- **Timeout per execution:** 3 seconds
- **Total samples processed:** 550
- **Total execution time:** 7.7 minutes (CPU)
- **Random seed:** 42

5 Results

5.1 Standalone Detection

Using Gradient Boosting on absolute trace features (12 features):

Key finding: Even standalone execution traces achieve 82.5% recall on boundary inversion (vs 0% static).

Detector	Accuracy	Precision	Recall	F1
XGBoost	0.751	0.856	0.838	0.846
Random Forest	0.693	0.875	0.729	0.795
Gradient Boosting	0.787	0.822	0.944	0.879
Logistic Regression	0.467	0.863	0.413	0.558

Table 3: Standalone detection performance.

Metric	Pairwise	Import-Aware	Hybrid
Accuracy	0.898	0.902	0.904
Precision	0.918	0.922	0.928
Recall	0.962	0.961	0.957
F1 Score	0.939	0.941	0.942

Table 4: Detection performance across three approaches.

5.2 Pairwise Detection

5.3 Per-Strategy Results

Strategy	Static (Paper 1)	Pairwise	Import-Aware
Boundary inversion	0.000	0.875	0.725
Import aliasing	0.070	0.133	0.582
Variable shadowing	1.000	0.969	0.959
Dead code	1.000	0.980	0.990
Comment planting	1.000	0.969	0.929

Table 5: Per-strategy recall comparison.

5.4 Statistical Validation

We validate our results using two statistical tests:

McNemar’s test: Compares paired predictions. $\chi^2 = 113.00$, $p = 0.0466$. Significant at $\alpha = 0.05$.

Wilcoxon signed-rank test: Compares per-sample accuracy distributions. Statistic = 476.00, $p = 0.0074$. Significant at $\alpha = 0.01$.

Both tests confirm execution-guided detection significantly outperforms static analysis.

6 Discussion

6.1 Why Execution Traces Work

Static analysis captures code structure; execution traces capture code behavior. Semantic perturbations preserve structure but change behavior—exactly what execution traces observe.

Example: `n % 2 == 0` vs `n % 2 == 1` (boundary inversion).

- **AST features:** Identical structure (both are comparison expressions)
- **CodeBERT embeddings:** Nearly identical vectors (one token difference)
- **Execution traces:** 90% of test inputs produce different outputs

6.2 Tradeoffs Between Perturbation Types

Different trace features help different perturbation types:

- **Output difference ratio:** Helps boundary inversion (90% output difference)
- **Import-aware features:** Help import aliasing (58.2% recall)
- **No single feature set** is optimal for all strategies

6.3 The Hybrid Approach

Combining execution traces with AST features achieves the best overall performance ($F1 = 0.942$). This suggests:

- AST features capture structural perturbations (100% recall)
- Execution traces capture semantic perturbations (87.5% recall)
- The combination covers both

6.4 Limitations

- **Execution overhead:** Sandbox execution is slower than static analysis (7.7 min for 550 samples)
- **Test input coverage:** May not trigger all code paths
- **Python-only:** Multi-language support requires adaptation
- **Sample size:** 550 samples is modest; larger datasets needed for stronger claims

6.5 Future Work

- Extend to JavaScript, Java, C++ code
- Use symbolic execution for smarter test input generation
- Combine with fuzzing for better branch coverage
- Scale to 10,000+ samples
- Real-world deployment testing

7 Conclusion

Execution-guided detection solves the semantic perturbation gap identified in prior work. Boundary inversion detection improves from 0% to 87.5%, and hybrid approaches achieve 0.942 F1. This demonstrates that observing code behavior—not just structure—is essential for robust adversarial code detection.

Our work establishes a new direction in AI code security: **execution-guided detection** as a complement to static analysis. As AI code generation becomes ubiquitous, robust detection of adversarial perturbations becomes critical. Execution-guided methods provide a path forward.

Reproducibility Statement

All code is available at <https://github.com/amanmukati09/execution-guided-detection>. The pipeline requires Python 3.14, GCC, and standard ML libraries. The dataset is publicly available from our prior work [1]. Random seed 42 is used throughout for reproducibility.

References

- [1] A. Mukati. Detecting adversarial perturbations in AI-generated code: A three-tier detectability hierarchy. Zenodo, 2026. DOI: 10.5281/zenodo.22349074.
- [2] A. Mukati. Adversarial code review: Can AI reviewers be fooled by semantic perturbations? Zenodo, 2026. DOI: 10.5281/zenodo.22357507.
- [3] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri. Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions. In *IEEE Symposium on Security and Privacy (S&P)*, 2022.
- [4] M. Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] R. Jia and P. Liang. Adversarial examples for evaluating reading comprehension systems. In *EMNLP*, 2017.
- [6] Z. Yang et al. Natural adversarial examples for code. In *Findings of ACL*, 2022.
- [7] Z. Feng et al. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of EMNLP*, 2020.